PYTHON · LIVE CODING · INTERVIEW PREP

جزوه‌ی فشرده‌ی Live Coding پایتون

هر چیزی که برای یک «small live coding task in Python» لازم داری — سریع بخوان، الگوها را یاد بگیر، با اعتمادبه‌نفس کد بزن.

40
مسئله و تمرین با راه‌حل کامل

100%
کدها واقعاً اجرا و تست شده‌اند؛ خروجی‌ها واقعی‌اند

12
بخش، از روش حل تا pandas و تمرین شبیه‌سازی

مصاحبه: Werkstudent*in Projektmanagement & Entwicklung — Mercedes-Benz

تهیه‌شده برای: احمدرضا نوروزی · اکتبر 2026

نسخه‌ی پایتون استفاده‌شده برای اجرای کدها: Python 3.13 · pandas 3.0 (همه‌ی کدها با Python 3.9+ هم کار می‌کنند)

START HERE

این جزوه را چطور بخوانی؟

این جزوه فقط برای **یک تسک کوچک لایو کدینگ** نوشته شده است، نه برای مصاحبه‌های سنگین الگوریتمی. برای نقش Werkstudent، سوال معمولاً 15 تا 30 دقیقه طول می‌کشد و سطحش ساده تا متوسط است: کار با رشته، لیست و دیکشنری، یک کلاس کوچک، یا خلاصه کردن یک دیتای کوچک. هر کد این جزوه اجرا شده و خروجی‌ای که می‌بینی **خروجی واقعی** همان کد است.

اگر فقط 30 دقیقه وقت داری:

1. بخش 1: روش حل و جمله‌های انگلیسی (7 دقیقه)
2. نگاه سریع به بخش 2: جعبه‌ابزار (8 دقیقه)
3. فقط این 8 مسئله را بخوان (10 دقیقه): P01، P02، P04، P06، P11، P16، P18 و P30
4. بخش 11.4: اشتباهات رایج (5 دقیقه)

اگر 3 ساعت وقت داری:

1. کل جزوه را یک‌بار سریع بخوان (1 ساعت).
2. 12 الگوی چک‌لیست آخر جزوه را **بدون نگاه کردن** در ادیتور تایپ کن و اجرا کن (1 ساعت).
3. تمرین‌های بخش 12 را با تایمر 20 دقیقه‌ای و **بلند و به انگلیسی** حل کن (1 ساعت).

01	روش حل مسئله در لایو کدینگ + جمله‌های انگلیسی آماده	3
02	جعبه‌ابزار پایتون Cheat Sheet: رشته، لیست، دیکشنری، set، collections	6
03	تابع، ورودی و مدیریت خطا	12
04	شیءگرایی (OOP) به اندازه‌ی لازم	16
05	کار با فایل JSON، CSV، TXT	20
06	مسئله‌های رشته P01 تا P10	22
07	مسئله‌های عددی P11 تا P15	27
08	مسئله‌های لیست P16 تا P28	30
09	پردازش داده با پایتون خالص P29 تا P35 — مرتبط‌ترین بخش با این نقش	37
10	pandas در 10 دقیقه	41
11	تست، Big-O و اشتباهات رایج	45
12	پنج تمرین شبیه‌سازی مصاحبه + چک‌لیست روز مصاحبه	49

PART 01

روش حل مسئله در لایو کدینگ

در لایو کدینگ، مصاحبه‌گر بیشتر از «جواب نهایی» به **روش فکر کردن** تو نگاه می‌کند. یک راه‌حل ساده که درست کار می‌کند و خوب توضیح داده شده، از یک راه‌حل «باهوشانه» که نیمه‌کاره مانده خیلی بهتر است.

1.1 مصاحبه‌گر به چه چیزهایی نمره می‌دهد؟

معیار	یعنی چه؟
ارتباط (Communication)	بلند فکر می‌کنی، سوال می‌پرسی، تصمیم‌هایت را توضیح می‌دهی.
درستی (Correctness)	کد برای مثال‌های عادی و حالت‌های مرزی (edge cases) درست کار می‌کند.
خوانایی (Readability)	اسم‌های معنی‌دار، تابع‌های کوچک، بدون کد تکراری.
تست (Testing)	خودت کدت را با چند مثال اجرا و چک می‌کنی، قبل از اینکه کسی بگوید.
پذیرش راهنمایی (Coachability)	وقتی hint می‌دهند، گوش می‌دهی و سریع از آن استفاده می‌کنی.

1.2 روش 6 مرحله‌ای (همیشه همین را انجام بده)

1. **بفهم و بپرس (Clarify) — 1 تا 2 دقیقه.** ورودی دقیقاً چیست؟ خروجی چه شکلی است؟ لیست خالی؟ عدد منفی؟ حروف بزرگ و کوچک؟ اگر چیزی مبهم است، یک فرض منطقی بکن و **بلند بگو**.
2. **مثال بزن (Examples).** یک مثال عادی و یک مثال مرزی را به شکل کامنت یا `assert` بنویس. این کار هم فهمت را نشان می‌دهد، هم بعداً تستت می‌شود.
3. **اول ایده‌ی ساده (Plan).** با زبان ساده بگو چه می‌خواهی بکنی. اگر راه بهینه به ذهنت نمی‌رسد، با راه ساده (brute force) شروع کن و بگو بعداً بهترش می‌کنی.
4. **کد بزن (Code).** یک تابع با اسم خوب بنویس و حین نوشتن توضیح بده. منطق اضافه را در یک تابع کمکی (helper function) بگذار.
5. **تست کن (Test).** کد را با مثال‌هایت اجرا کن. حالت‌های مرزی را چک کن: ورودی خالی، یک عضو، مقادیر تکراری، `None` .
6. **پیچیدگی و بهبود (Big-O & Optimize).** پیچیدگی زمانی و حافظه را بگو. اگر وقت ماند، بهترش کن یا بگو چطور بهترش می‌کردی.

1.3 جمله‌های انگلیسی آماده

What to say	موقعیت
Let me make sure I understand the task. The input is …, and I should return …, right?	شروع
Can I assume the input is always a list of integers? What should happen if it's empty?	سوال
I'll assume the names are case-insensitive. Is that okay?	فرض کردن
Let me write a quick example: for [2, 7, 11] and target 9, I expect (0, 1).	مثال
My first idea is a brute-force solution with two loops. It's O(n²), but it's a good start.	ایده‌ی ساده
I can improve this with a dictionary, so the lookup is O(1) and the whole thing is O(n).	بهینه کردن
I'll move this part into a small helper function to keep it readable.	حین کد زدن
Let me test it with the normal case and a few edge cases, like an empty list.	تست
Hmm, that's not what I expected. Let me trace through the loop step by step.	باگ
I don't remember the exact method name. Normally I'd check the docs, so I'll write it like this for now.	یادم نیست
Good point, thank you. So if I use a set here, the membership check becomes O(1).	بعد از hint
The time complexity is O(n) and the space is O(n). With more time, I'd add input validation and unit tests.	پایان

English	اصطلاح	English	اصطلاح
iterate over / loop through	پیمایش کردن	edge case	حالت مرزی
immutable	تغییرناپذیر	nested loop	حلقه‌ی تودرتو
in place	در همان جا (بدون کپی)	time complexity	پیچیدگی زمانی
trade-off	بده‌بستان / مصالحه	brute force	راه‌حل ساده و کند
recursive	بازگشتی	dictionary / hash map lookup	جست‌وجو در دیکشنری

1.4 یک مثال کامل با «بلند فکر کردن»

Task: Given a list of trips as (driver, km), return the n longest trips, longest first.

- **Clarify:** «اگر دو سفر طول برابر داشتند چه؟ اگر لیست کمتر از n عضو داشت؟» ← فرض: ترتیب اولیه‌ی سفرهای برابر حفظ شود؛ اگر کمتر بود، همه را برگردان.
- **Example:** ورودی `[("Ali", 12.5), ("Sara", 40.0), ("Mia", 7.2), ("Tom", 40.0)]` با `n=2` ← خروجی Sara و بعد Tom.
- **Plan:** «بر اساس km نزولی مرتب می‌کنم و n تای اول را برمی‌دارم.»
- **Code + Test:**

```python
def top_n_trips(trips, n=3):
    """Return the n longest trips as (driver, km), longest first."""
    if n <= 0:
        return []
    return sorted(trips, key=lambda t: t[1], reverse=True)[:n]

trips = [("Ali", 12.5), ("Sara", 40.0), ("Mia", 7.2), ("Tom", 40.0)]
assert top_n_trips(trips, 2) == [("Sara", 40.0), ("Tom", 40.0)]
assert top_n_trips([], 3) == []
assert top_n_trips(trips, 0) == []
print(top_n_trips(trips))
```

```
OUTPUT
[('Sara', 40.0), ('Tom', 40.0), ('Ali', 12.5)]
```

"Sorting is O(n log n). Python's sort is stable, so trips with the same distance keep their original order. If the list were very long and I needed only a few trips, heapq.nlargest would give O(N log n) instead of O(N log N)."

1.5 نکته‌های محیط مصاحبه

- اگر نمی‌دانی کجا کد می‌زنی (VS Code، HackerRank، CoderPad با اشتراک صفحه، Jupyter)، قبل از مصاحبه با یک ایمیل کوتاه بپرس. همین‌طور بپرس آیا گشتن در مستندات (Google / docs) مجاز است.
- پایتون، VS Code و Jupyter را روی لپ‌تاپ آماده کن، فونت ادیتور را بزرگ کن و نوتیفیکیشن‌ها را خاموش کن.
- اگر محیط اجرای کد ندارد (مثلاً یک Google Doc)، کد را با دست trace کن: «for i = 0, value is 2 ...».
- در ایمیل گفته‌اند استفاده از AI برای تسک خانگی مجاز است. در لایو کدینگ معمولاً این‌طور نیست، پس الگوهای این جزوه را خودت باید بتوانی بنویسی.

PART 02

جعبه‌ابزار پایتون (Cheat Sheet)

این بخش چیزهایی را دارد که در 90٪ تسک‌های کوچک لازم می‌شوند. جلوی هر خط با رنگ سبز، **مقدار واقعی** آن عبارت آمده است (`# → …`).

2.1 رشته‌ها (str)

```
s = "  Mercedes-Benz EQS  "
s.strip()                          # → 'Mercedes-Benz EQS'
s.strip().lower()                  # → 'mercedes-benz eqs'
s.strip().replace("-", " ")        # → 'Mercedes Benz EQS'
s.split()                          # → ['Mercedes-Benz', 'EQS']
"a,b,,c".split(",")                # → ['a', 'b', '', 'c']
"-".join(["2026", "10", "18"])     # → '2026-10-18'
"EQS 580".startswith("EQ")         # → True
"EQS 580".find("580")              # → 4
"EQS 580".find("x")                # → -1
"banana".count("a")                # → 3
"hello world".title()              # → 'Hello World'
"abc123".isalnum(), "123".isdigit(), "abc".isalpha()  # → (True, True, True)
```

نکته: رشته‌ها immutable هستند: هر متد یک رشته‌ی **جدید** برمی‌گرداند و رشته‌ی اصلی عوض نمی‌شود. پس باید بنویسی `s = s.strip()`. تفاوت `split()` بدون آرگومان: فاصله‌های اضافه را خودش حذف می‌کند.

2.2 f-string و فرمت عدد

```
model, price, km = "EQS", 109990.5, 12345
f"{model} costs {price:,.2f} EUR"  # → 'EQS costs 109,990.50 EUR'
f"{km:>8}|"                        # → '   12345|'
f"{km:<8}|"                        # → '12345   |'
f"{0.4567:.1%}"                    # → '45.7%'
f"{7:03d}"                         # → '007'
f"{model=}"                        # → "model='EQS'"
```

Spec	معنی	Spec	معنی
`:.2f`	دو رقم اعشار	`:^8` / `:<8` / `:>8`	راست‌چین / چپ‌چین / وسط‌چین در 8 خانه
`:03d`	صفر در ابتدا تا 3 رقم	`:,`	جداکننده‌ی هزارگان
`:.1%`	نمایش درصد	`{x=}`	اسم و مقدار (برای دیباگ)

2.3 Slicing

قاعده: `seq[start:stop:step]` — خانه‌ی `stop` شامل **نمی‌شود**. اندیس منفی از آخر می‌شمارد (`-1` یعنی آخرین عضو). روی رشته، لیست و tuple یکسان کار می‌کند.

```
s = "Python"
s[0], s[-1]   # → ('P', 'n')
s[1:4]        # → 'yth'
s[:2], s[2:]  # → ('Py', 'thon')
s[::-1]       # → 'nohtyP'
s[::2]        # → 'Pto'
nums = [10, 20, 30, 40, 50]
nums[-2:]     # → [40, 50]
nums[1:-1]    # → [20, 30, 40]
```

2.4 لیست (list)

```
cars = ["C", "E", "S"]
cars.append("G")
cars.insert(0, "A")
cars                        # → ['A', 'C', 'E', 'S', 'G']
cars.extend(["EQA", "EQB"])
cars.pop()                  # → 'EQB'
cars.pop(0)                 # → 'A'
cars.remove("E")
cars                        # → ['C', 'S', 'G', 'EQA']
"S" in cars                 # → True
cars.index("S")             # → 1
len(cars), cars.count("C")  # → (4, 1)
```

2.5 مرتب‌سازی و `key`

```
nums = [5, 2, 9, 1]
sorted(nums)                       # → [1, 2, 5, 9]
sorted(nums, reverse=True)         # → [9, 5, 2, 1]
nums                               # → [5, 2, 9, 1]
nums.sort()
nums                               # → [1, 2, 5, 9]
words = ["banana", "apple", "Fig"]
sorted(words)                      # → ['Fig', 'apple', 'banana']
sorted(words, key=str.lower)       # → ['apple', 'banana', 'Fig']
sorted(words, key=len)             # → ['Fig', 'apple', 'banana']
cars = [("EQS", 2022), ("C", 2014), ("GLE", 2019)]
sorted(cars, key=lambda c: c[1])  # → [('C', 2014), ('GLE', 2019), ('EQS', 2022)]
max(cars, key=lambda c: c[1])     # → ('EQS', 2022)
```

حواست باشد: `sorted(x)` یک لیست **جدید** برمی‌گرداند، اما `x.sort()` خود لیست را تغییر می‌دهد و `None` برمی‌گرداند. پس هرگز ننویس `nums = nums.sort()` . حروف بزرگ قبل از کوچک مرتب می‌شوند، برای همین `key=str.lower` لازم است.

Comprehension 2.6

```
nums = [1, 2, 3, 4, 5, 6]
[n * n for n in nums]              # → [1, 4, 9, 16, 25, 36]
[n for n in nums if n % 2 == 0]    # → [2, 4, 6]
["even" if n % 2 == 0 else "odd" for n in nums[:3]]  # → ['odd', 'even', 'odd']
{n: n * n for n in nums[:3]}       # → {1: 1, 2: 4, 3: 9}
{len(w) for w in ["a", "bb", "cc"]}  # → {1, 2}
[(i, j) for i in range(2) for j in range(2)]  # → [(0, 0), (0, 1), (1, 0), (1, 1)]
sum(n for n in nums if n > 3)      # → 15
```

نکته: فیلتر (`if` تنها) **بعد** از `for` می‌آید، ولی `if/else` (انتخاب مقدار) **قبل** از `for` . اگر comprehension از یک خط بیشتر شد، یک حلقه‌ی معمولی بنویس؛ خوانایی مهم‌تر است.

دیکشنری (dict) 2.7

```
car = {"model": "EQS", "year": 2022}
car["model"]               # → 'EQS'
car.get("color")           # → None
car.get("color", "black")  # → 'black'
car["color"] = "silver"
"year" in car              # → True
list(car.keys())           # → ['model', 'year', 'color']
list(car.values())         # → ['EQS', 2022, 'silver']
car.pop("color")           # → 'silver'
car.update({"km": 12000})
car                        # → {'model': 'EQS', 'year': 2022, 'km': 12000}
car.setdefault("options", []).append("AMG Line")
car
    # → {'model': 'EQS', 'year': 2022, 'km': 12000, 'options': ['AMG Line']}
```

```
prices = {"C": 48_000, "E": 62_000, "S": 115_000}
for model, price in prices.items():
    print(f"{model}: {price}")

cheapest = min(prices, key=prices.get)       # key with the smallest value
print("cheapest:", cheapest)
by_price = sorted(prices.items(), key=lambda kv: kv[1], reverse=True)
print(by_price)
```

```
OUTPUT
C: 48000
E: 62000
S: 115000
cheapest: C
[('S', 115000), ('E', 62000), ('C', 48000)]
```

الگوی شمارش را حفظ کن؛ تقریباً در نصف مسئله‌ها لازم می‌شود:

```
words = "red blue red green blue red".split()
counts = {}
for w in words:
    counts[w] = counts.get(w, 0) + 1      # the classic counting pattern
print(counts)
```

```
OUTPUT
{'red': 3, 'blue': 2, 'green': 1}
```

نکته: `d[key]` اگر کلید نباشد `KeyError` می‌دهد، ولی `d.get(key, default)` خطا نمی‌دهد. از پایتون 3.7 به بعد، دیکشنری ترتیب اضافه شدن کلیدها را حفظ می‌کند.

2.8 مجموعه (set)

```
a = {1, 2, 3, 4}
b = {3, 4, 5}
a | b                 # → {1, 2, 3, 4, 5}
a & b                 # → {3, 4}
a - b                 # → {1, 2}
a ^ b                 # → {1, 2, 5}
set([1, 1, 2, 2, 3])  # → {1, 2, 3}
3 in a                # → True
```

نکته: چک کردن `x in my_set` یا `x in my_dict` به طور میانگین `O(1)` است، ولی `x in my_list` برابر `O(n)` است. set ترتیب ندارد و عضو تکراری نمی‌گیرد.

2.9 Tuple و Unpacking

```
x, y = (3, 4)
x, y          # → (3, 4)
a, b = 1, 2
a, b = b, a
a, b          # → (2, 1)
first, *rest = [1, 2, 3, 4]
first, rest   # → (1, [2, 3, 4])
q, r = divmod(17, 5)
q, r          # → (3, 2)
```

2.10 توابع built-in مهم

```
models = ["C", "E", "S"]
years = [2014, 2016, 2020]
for i, m in enumerate(models, start=1):
    print(i, m)
for m, y in zip(models, years):
    print(m, y)
print(dict(zip(models, years)))
print(any(y > 2019 for y in years), all(y > 2019 for y in years))
print(list(range(0, 10, 3)), sum(years) / len(years))
print(round(3.14159, 2), abs(-7), 7 // 2, 7 % 2, 2 ** 10)
```

```
OUTPUT
1 C
2 E
3 S
C 2014
E 2016
S 2020
{'C': 2014, 'E': 2016, 'S': 2020}
True False
[0, 3, 6, 9] 2016.6666666666667
3.14 7 3 1 1024
```

`enumerate` وقتی اندیس و مقدار را با هم می‌خواهی، `zip` برای پیمایش هم‌زمان دو لیست، `any/all` برای «آیا حداقل یکی / همه شرط را دارند؟». `//` تقسیم صحیح، `%` باقی‌مانده، `**` توان.

2.11 ماژول `collections`

```
from collections import Counter
c = Counter("mississippi")
c                                   # → Counter({'i': 4, 's': 4, 'p': 2, 'm': 1})
c.most_common(2)                    # → [('i', 4), ('s', 4)]
c["s"], c["z"]                      # → (4, 0)
Counter(["EQS", "C", "EQS"]) + Counter(["C"])  # → Counter({'EQS': 2, 'C': 2})
```

```
from collections import defaultdict

cars = [("EQS", "electric"), ("C200", "petrol"), ("EQE", "electric")]
groups = defaultdict(list)          # missing key -> starts as []
for model, fuel in cars:
    groups[fuel].append(model)
print(dict(groups))

totals = defaultdict(int)           # missing key -> starts as 0
for model, fuel in cars:
    totals[fuel] += 1
print(dict(totals))
```

```
OUTPUT
{'electric': ['EQS', 'EQE'], 'petrol': ['C200']}
{'electric': 2, 'petrol': 1}
```

```python
stack = [1, 2, 3]                    # list = stack (LIFO)
stack.append(4)
stack.pop()  # → 4
from collections import deque
q = deque([1, 2, 3])                 # deque = queue (FIFO)
q.append(4)
q.popleft()  # → 1
q            # → deque([2, 3, 4])
```

نکته: stack (آخرین ورودی، اولین خروجی) = لیست با `append/pop` . queue (اولین ورودی، اولین خروجی) = `deque` با `append/popleft` ؛ چون `list.pop(0)` کند است (O(n)).

2.12 تبدیل نوع و Truthiness

```python
int("42") + 1                        # → 43
float("3.5")                         # → 3.5
str(2026) + "!"                      # → '2026!'
list("abc")                          # → ['a', 'b', 'c']
int("ff", 16), bin(5)                # → (255, '0b101')
type(3.0).__name__                   # → 'float'
isinstance(3, (int, float))          # → True
bool(""), bool([]), bool(0), bool("0")  # → (False, False, False, True)
```

نکته: مقادیر «falsy»: `0` ، `0.0` ، `""` ، `[]` ، `{}` ، `set()` ، `None` ، `False` . پس `if not items:` یعنی «اگر لیست خالی است». ولی رشته‌ی `"0"` یا `"False"` truthy است!

PART 03

تابع، ورودی و مدیریت خطا

3.1 تعریف تابع، type hint و docstring

```python
def average(values: list[float]) -> float:
    """Return the arithmetic mean of values."""
    if not values:
        raise ValueError("values must not be empty")
    return sum(values) / len(values)

def min_max(values):
    return min(values), max(values)          # returns a tuple

print(average([2, 4, 6]))
lo, hi = min_max([3, 9, 1])
print(lo, hi)
```

```
OUTPUT
4.0
1 9
```

type hint (`list[float]`، `-> float`) اجباری نیست و پایتون آن را چک نمی‌کند، ولی کد را حرفه‌ای‌تر و خواناتر می‌کند. وقتی چند مقدار `return` می‌کنی، در واقع یک tuple برمی‌گردانی.

3.2 آرگومان پیش‌فرض، `*args` و `**kwargs`

```python
def greet(name, greeting="Hello"):           # default argument
    return f"{greeting}, {name}!"

def total(*args):                            # any number of positional args
    return sum(args)

def describe(**kwargs):                      # any number of keyword args
    return ", ".join(f"{k}={v}" for k, v in kwargs.items())

print(greet("Nico"))
print(greet("Nico", greeting="Hi"))
print(total(1, 2, 3))
print(describe(model="EQS", year=2022))
```

```
OUTPUT
Hello, Nico!
Hi, Nico!
6
model=EQS, year=2022
```

3.3 `lambda` ، `map` و `filter`

```
square = lambda x: x * x
square(5)                           # → 25
list(map(str.upper, ["a", "b"]))    # → ['A', 'B']
list(filter(lambda n: n > 2, [1, 2, 3, 4]))  # → [3, 4]
[n for n in [1, 2, 3, 4] if n > 2]  # → [3, 4]
```

lambda یک تابع یک‌خطی بدون اسم است؛ بیشتر به عنوان `key=` در `sorted/min/max` به کار می‌رود. در پایتون، comprehension (خط آخر) معمولاً از `map/filter` خواناتر است.

3.4 گرفتن ورودی از کاربر

```
raw = input("Enter mileage: ")              # input() ALWAYS returns a str
try:
    km = int(raw)
    print(f"You entered {km} km")
except ValueError:
    print("Please enter a whole number")
```

```
OUTPUT
Enter mileage: 42
You entered 42 km
```

(در این اجرا، ورودی کاربر شبیه‌سازی شده و مقدار 42 وارد شده است.)

3.5 `try / except / else / finally`

```
def safe_divide(a, b):
    try:
        result = a / b
    except ZeroDivisionError:
        print("cannot divide by zero")
        return None
    except TypeError as e:
        print("bad input:", e)
        return None
    else:                                   # runs only if no exception
        return result
    finally:                                # always runs
        print("done")

print(safe_divide(10, 2))
print(safe_divide(1, 0))
```

```
OUTPUT
done
5.0
cannot divide by zero
done
None
```

به ترتیب خروجی دقت کن: `finally` **قبل** از اینکه مقدار برگشتی چاپ شود اجرا می‌شود، چون همیشه موقع خروج از تابع اجرا می‌شود.

3.6 الگوی «ورودی کثیف را تمیز کن، کرش نکن»

```python
def parse_numbers(items):
    """Convert strings to floats and collect the invalid ones."""
    numbers, errors = [], []
    for item in items:
        try:
            numbers.append(float(item))
        except (ValueError, TypeError):
            errors.append(item)
    return numbers, errors

print(parse_numbers(["1.5", "abc", "3", None, ""]))
```

```
OUTPUT
([1.5, 3.0], ['abc', None, ''])
```

3.7 `raise` و Exception سفارشی

```python
class InvalidVinError(Exception):
    """Raised when a vehicle identification number is invalid."""

def validate_vin(vin: str) -> str:
    vin = vin.strip().upper()
    if len(vin) != 17:
        raise InvalidVinError(f"VIN must have 17 chars, got {len(vin)}")
    if any(ch in "IOQ" for ch in vin):
        raise InvalidVinError("VIN must not contain I, O or Q")
    return vin

for v in ["wdd2220011a123456", "ABC"]:
    try:
        print("valid:", validate_vin(v))
    except InvalidVinError as e:
        print("error:", e)
```

```
OUTPUT
valid: WDD2220011A123456
error: VIN must have 17 chars, got 3
```

Exception	Example	کی رخ می‌دهد؟
`ValueError`	`int("abc")`	نوع درست است ولی مقدار نامعتبر است
`TypeError`	`"a" + 1`	نوع اشتباه است
`KeyError`	`d["missing"]`	کلید در دیکشنری نیست
`IndexError`	`[1, 2][5]`	اندیس خارج از محدوده است
`AttributeError`	`None.upper()`	متد یا ویژگی وجود ندارد (معمولاً یک None غیرمنتظره)
`ZeroDivisionError`	`1 / 0`	تقسیم بر صفر
`FileNotFoundError`	`open("x.csv")`	فایل پیدا نشد

سوال احتمالی: "Why is a bare `except:` a bad idea?" — چون همه‌چیز را می‌گیرد (حتی KeyboardInterrupt و باگ‌های خودت) و خطا را پنهان می‌کند. همیشه Exception مشخص را بگیر: `except ValueError:` .

PART 04

شیءگرایی (OOP) به اندازه‌ی لازم

سوال OOP در یک تسک کوچک معمولاً این شکلی است: «یک کلاس Car / Task / BankAccount بساز که چند ویژگی، چند متد و کمی اعتبارسنجی داشته باشد.» همین چهار الگو کافی است.

4.1 کلاس، `__init__`، `self` و متد

```python
class Car:
    wheels = 4                                  # class attribute (shared)

    def __init__(self, model: str, year: int, km: float = 0):
        self.model = model                      # instance attributes
        self.year = year
        self.km = km

    def drive(self, distance: float) -> None:
        if distance < 0:
            raise ValueError("distance must be positive")
        self.km += distance

    def needs_service(self, interval: int = 25_000) -> bool:
        return self.km >= interval

    def __repr__(self):                         # how the object is printed
        return f"Car({self.model!r}, {self.year}, km={self.km})"

c = Car("C200", 2021)
c.drive(30_000)
print(c, c.needs_service(), Car.wheels)
```

```
OUTPUT
Car('C200', 2021, km=30000) True 4
```

- `__init__` موقع ساختن شیء اجرا می‌شود؛ `self` یعنی «همین شیء».
- **class attribute** (مثل `wheels`) بین همه‌ی شیءها مشترک است؛ **instance attribute** (مثل `self.km`) مال هر شیء جداگانه است.
- `__repr__` تعیین می‌کند شیء موقع `print` چطور نمایش داده شود. بدون آن چیزی مثل `<Car object at 0x…>` می‌بینی.

4.2 ارث‌بری و `super()`

```python
class ElectricCar(Car):
    def __init__(self, model, year, battery_kwh, km=0):
        super().__init__(model, year, km)    # reuse the parent __init__
        self.battery_kwh = battery_kwh

    def range_km(self, kwh_per_100km=18):
        return round(self.battery_kwh / kwh_per_100km * 100)

    def __repr__(self):                      # override the parent method
        return f"ElectricCar({self.model!r}, {self.battery_kwh} kWh)"

eqs = ElectricCar("EQS", 2022, 108)
eqs.drive(100)                               # inherited from Car
print(eqs, eqs.range_km(), eqs.km, isinstance(eqs, Car))
```

```
OUTPUT
ElectricCar('EQS', 108 kWh) 600 100 True
```

4.3 ترکیب (Composition): کلاسی که لیستی از شیءها را مدیریت می‌کند

```python
class Fleet:
    def __init__(self):
        self.cars = []                       # a Fleet HAS cars

    def add(self, car):
        self.cars.append(car)

    def __len__(self):
        return len(self.cars)

    def total_km(self):
        return sum(c.km for c in self.cars)

    def due_for_service(self, interval=25_000):
        return [c.model for c in self.cars if c.needs_service(interval)]

fleet = Fleet()
for model, year, km in [("C200", 2020, 40_000), ("E300", 2022, 9_000),
                        ("S500", 2019, 61_000)]:
    fleet.add(Car(model, year, km))
print(len(fleet), fleet.total_km(), fleet.due_for_service())
```

```
OUTPUT
3 110000 ['C200', 'S500']
```

قاعده: رابطه‌ی «is-a» (خودروی برقی **یک** خودرو است) ← ارث‌بری. رابطه‌ی «has-a» (ناوگان **تعدادی** خودرو دارد) ← ترکیب. در تسک‌های واقعی، ترکیب بیشتر به کار می‌آید.

4.4 `@property` برای اعتبارسنجی

```python
class Battery:
    def __init__(self, level=100):
        self.level = level                    # goes through the setter

    @property
    def level(self):
        return self._level

    @level.setter
    def level(self, value):
        if not 0 <= value <= 100:
            raise ValueError("level must be between 0 and 100")
        self._level = value

b = Battery(80)
b.level = 55
print(b.level)
try:
    b.level = 120
except ValueError as e:
    print("error:", e)
```

```
OUTPUT
55
error: level must be between 0 and 100
```

4.5 `@dataclass` : سریع‌ترین راه برای کلاس داده

```python
from dataclasses import dataclass, field

@dataclass
class Task:
    title: str
    owner: str
    hours: float = 0.0
    done: bool = False
    tags: list = field(default_factory=list)  # never use tags: list = []

t = Task("Write report", "Ali", 3)
t.tags.append("docs")
print(t)
print(t == Task("Write report", "Ali", 3, tags=["docs"]))  # __eq__ is free
```

```
OUTPUT
Task(title='Write report', owner='Ali', hours=3, done=False, tags=['docs'])
True
```

```python
from dataclasses import dataclass

@dataclass(order=True)                    # adds <, > based on fields
class Version:
    major: int
    minor: int

versions = [Version(2, 1), Version(1, 9), Version(2, 0)]
print(sorted(versions)[0], max(versions))
```

```
OUTPUT
Version(major=1, minor=9) Version(major=2, minor=1)
```

Method	کی صدا زده می‌شود؟	Method	کی صدا زده می‌شود؟
`__init__`	`Car(...)`	`__len__`	`len(obj)`
`__repr__` / `__str__`	`print(obj)`	`__eq__`	`a == b`
`__lt__`	`sorted()` و `a < b`	`__contains__`	`x in obj`

سوال‌های احتمالی:

- "What are the main principles of OOP?" — **Encapsulation** (داده و متد کنار هم، با کنترل دسترسی)، **Inheritance** (ارث‌بری)، **Polymorphism** (یک متد با رفتار متفاوت در کلاس‌های مختلف، مثل `__repr__` بالا)، **Abstraction** (پنهان کردن جزئیات).
- "@staticmethod vs @classmethod?" — staticmethod نه `self` می‌گیرد نه `cls` (یک تابع معمولی داخل کلاس است). classmethod `cls` می‌گیرد و معمولاً برای سازنده‌ی جایگزین به کار می‌رود، مثل `Car.from_dict(d)` .

PART 05

کار با فایل: TXT، CSV، JSON

5.1 فایل متنی و `with`

```python
with open("notes.txt", "w", encoding="utf-8") as f:     # "w" overwrites
    f.write("first line\nsecond line\n")

with open("notes.txt", "a", encoding="utf-8") as f:     # "a" appends
    f.write("third line\n")

with open("notes.txt", encoding="utf-8") as f:          # "r" is default
    for number, line in enumerate(f, start=1):
        print(number, line.strip())                     # strip the "\n"
```

```
OUTPUT
1 first line
2 second line
3 third line
```

`with` فایل را خودش می‌بندد، حتی اگر وسط کار خطا رخ دهد. حالت‌ها: `"r"` خواندن (پیش‌فرض)، `"w"` نوشتن (محتوای قبلی پاک می‌شود)، `"a"` اضافه کردن به آخر فایل. همیشه `encoding="utf-8"` بنویس.

5.2 CSV با ماژول `csv`

```python
import csv

rows = [{"model": "C200", "price": "48000"},
        {"model": "EQS", "price": "109990"}]
with open("cars.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=["model", "price"])
    writer.writeheader()
    writer.writerows(rows)

with open("cars.csv", newline="", encoding="utf-8") as f:
    for row in csv.DictReader(f):            # each row is a dict
        print(row["model"], float(row["price"]))    # values are str!
```

```
OUTPUT
C200 48000.0
EQS 109990.0
```

حواست باشد: همه‌ی مقدارهایی که از CSV خوانده می‌شوند **رشته** هستند؛ باید خودت `int()` یا `float()` کنی. هنگام نوشتن، `newline=""` جلوی خط‌های خالی اضافه در ویندوز را می‌گیرد.

JSON 5.3

```python
import json

data = {"model": "EQS", "options": ["AMG Line", "Pano roof"], "km": 1200}
with open("car.json", "w", encoding="utf-8") as f:
    json.dump(data, f, indent=2)             # dict -> file

with open("car.json", encoding="utf-8") as f:
    loaded = json.load(f)                    # file -> dict
print(loaded["options"][0])

print(json.dumps({"a": 1, "ok": True}))      # dict -> str
print(json.loads('{"speed": 120}')["speed"]) # str -> dict
```

```
OUTPUT
AMG Line
{"a": 1, "ok": true}
120
```

یادت بماند: `dump/load` با **فایل** کار می‌کنند و `dumps/loads` با **رشته** (حرف s یعنی string).

`pathlib` 5.4 (روش مدرن کار با مسیرها)

```python
from pathlib import Path

p = Path("notes.txt")
print(p.exists(), p.suffix, p.stem)
print(p.read_text(encoding="utf-8").splitlines())
print(sorted(f.name for f in Path(".").glob("*.csv")))
```

```
OUTPUT
True .txt notes
['first line', 'second line', 'third line']
['cars.csv']
```

PART 06

مسئله‌های رشته

برای هر مسئله: صورت سوال، راه‌حل تست‌شده، پیچیدگی، و سوالی که احتمالاً مصاحبه‌گر بعدش می‌پرسد.

P01 برعکس کردن رشته Reverse a string

O(n)

راه پایتونی `s[::-1]` است. اگر گفتند «بدون slicing»، با حلقه بنویس.

```python
def reverse_string(s: str) -> str:
    return s[::-1]

def reverse_string_loop(s: str) -> str:
    chars = []
    for i in range(len(s) - 1, -1, -1):      # walk from the end
        chars.append(s[i])
    return "".join(chars)

assert reverse_string("abc") == reverse_string_loop("abc") == "cba"
assert reverse_string("") == ""
print(reverse_string("Mercedes"), "".join(reversed("Benz")))
```

```
OUTPUT
sedecreM zneB
```

سوال احتمالی: «چرا `result = ch + result` داخل حلقه خوب نیست؟» — چون رشته immutable است و هر بار یک رشته‌ی جدید ساخته می‌شود ← در مجموع $O(n^2)$. راه درست: کاراکترها را در یک لیست جمع کن و آخر سر `"".join()` بزن.

P02 پالیندروم Valid palindrome

O(n)

آیا رشته از دو طرف یکسان خوانده می‌شود؟ معمولاً باید حروف بزرگ/کوچک و علامت‌ها نادیده گرفته شوند (اول بپرس!).

```python
def is_palindrome(s: str) -> bool:
    cleaned = [ch.lower() for ch in s if ch.isalnum()]
    return cleaned == cleaned[::-1]

def is_palindrome_two_pointers(s: str) -> bool:
    left, right = 0, len(s) - 1
    while left < right:
        if not s[left].isalnum():
            left += 1
        elif not s[right].isalnum():
            right -= 1
        elif s[left].lower() != s[right].lower():
            return False
        else:
            left += 1
            right -= 1
    return True

for text in ["racecar", "A man, a plan, a canal: Panama", "hello", ""]:
    print(repr(text), is_palindrome(text), is_palindrome_two_pointers(text))
```

```
OUTPUT
'racecar' True True
'A man, a plan, a canal: Panama' True True
'hello' False False
'' True True
```

نسخه‌ی دوم با دو اشاره‌گر (two pointers) کار می‌کند و حافظه‌ی اضافه `O(1)` دارد. اگر مصاحبه‌گر پرسید «بهترش کن»، همین را بگو.

P03 شمردن حروف صدادار Count vowels

`O(n)`

```python
def count_vowels(s: str) -> int:
    return sum(1 for ch in s.lower() if ch in "aeiou")

assert count_vowels("Mercedes-Benz") == 4
assert count_vowels("") == 0
print(count_vowels("Hello World"))
```

```
OUTPUT
3
```

P04 فراوانی کاراکترها Character frequency

`O(n)`

اول با دیکشنری و `get` بنویس (نشان می‌دهد الگو را بلدی)، بعد بگو «In practice I'd use Counter».

```python
from collections import Counter

def char_frequency(s: str) -> dict:
    freq = {}
    for ch in s:
        freq[ch] = freq.get(ch, 0) + 1
    return freq

print(char_frequency("banana"))
print(Counter("banana"))
print(Counter("banana").most_common(1))      # most frequent character
```

```
OUTPUT
{'b': 1, 'a': 3, 'n': 2}
Counter({'a': 3, 'n': 2, 'b': 1})
[('a', 3)]
```

P05 اولین کاراکتر غیرتکراری First non-repeating character

O(n)

دو دور: دور اول بشمار، دور دوم اولین کاراکتری را که تعدادش 1 است پیدا کن.

```python
from collections import Counter

def first_unique_char(s: str):
    counts = Counter(s)                    # pass 1: count
    for i, ch in enumerate(s):             # pass 2: first with count 1
        if counts[ch] == 1:
            return i, ch
    return None

print(first_unique_char("swiss"))
print(first_unique_char("aabb"))
```

```
OUTPUT
(1, 'w')
None
```

P06 آناگرام Anagram check / group anagrams

O(n)

دو کلمه آناگرام‌اند اگر حروف یکسان با تعداد یکسان داشته باشند (listen / silent).

```python
from collections import Counter, defaultdict

def is_anagram(a: str, b: str) -> bool:
    a = a.replace(" ", "").lower()
    b = b.replace(" ", "").lower()
    return Counter(a) == Counter(b)          # or: sorted(a) == sorted(b)

def group_anagrams(words):
    groups = defaultdict(list)
    for w in words:
        groups["".join(sorted(w))].append(w) # same letters -> same key
    return list(groups.values())

print(is_anagram("Listen", "Silent"), is_anagram("abc", "abd"))
print(group_anagrams(["eat", "tea", "tan", "ate", "nat", "bat"]))
```

```
OUTPUT
True False
[['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]
```

`Counter` برابر `O(n)` است؛ `sorted(a) == sorted(b)` ساده‌تر است ولی `O(n log n)`. برای گروه‌بندی، «حروف مرتب‌شده» را کلید دیکشنری می‌کنیم.

P07 پرتکرارترین کلمه‌ها Top-k words

O(n log n)

متن را به حروف کوچک تبدیل کن، علامت‌ها را حذف کن، بشمار و k تای اول را برگردان.

```python
import re
from collections import Counter

def top_words(text: str, k: int = 3):
    words = re.findall(r"[a-z']+", text.lower())
    return Counter(words).most_common(k)

def top_words_plain(text: str, k: int = 3):
    counts = {}
    for w in text.lower().split():
        w = w.strip(".,!?;:\"'()")
        if w:
            counts[w] = counts.get(w, 0) + 1
    # sort by count (high first), then alphabetically for ties
    return sorted(counts.items(), key=lambda kv: (-kv[1], kv[0]))[:k]

text = "The car is fast. The car is red! Is the car electric?"
print(top_words(text))
print(top_words_plain(text))
```

```
OUTPUT
[('the', 3), ('car', 3), ('is', 3)]
[('car', 3), ('is', 3), ('the', 3)]
```

به حالت مساوی (tie) دقت کن: `most_common` ترتیب اولین دیدن را نگه می‌دارد، ولی با `key=lambda kv: (-kv[1], kv[0])` می‌توانی حالت مساوی را الفبایی کنی. علامت منفی یعنی «عدد را نزولی مرتب کن».

P08 کار با جمله Reverse words, capitalize, longest word

O(n)

```python
sentence = "  python is   fun  "
" ".join(sentence.split()[::-1])    # → 'fun is python'
" ".join(w.capitalize() for w in sentence.split())  # → 'Python Is Fun'
" ".join(w[::-1] for w in sentence.split())  # → 'nohtyp si nuf'
max(sentence.split(), key=len)      # → 'python'
```

P09 فشرده‌سازی رشته Run-length encoding

O(n)

`"aaabbc"` ← `"a3b2c1"` . اشتباه رایج: فراموش کردن آخرین گروه بعد از حلقه.

```python
from itertools import groupby

def compress(s: str) -> str:
    if not s:
        return ""
    result = []
    current, count = s[0], 1
    for ch in s[1:]:
        if ch == current:
            count += 1
        else:
            result.append(f"{current}{count}")
            current, count = ch, 1
```

```
    result.append(f"{current}{count}")       # don't forget the last group
    return "".join(result)

def compress_groupby(s: str) -> str:
    return "".join(f"{ch}{len(list(g))}" for ch, g in groupby(s))

assert compress("aaabbc") == compress_groupby("aaabbc") == "a3b2c1"
assert compress("") == ""
print(compress("aaabccdddd"))
```

OUTPUT

```
a3b1c2d4
```

سوال بعدی معمولاً این است: «برعکسش را بنویس»:

```
import re

def decompress(s: str) -> str:
    """'a3b2' -> 'aaabb'"""
    return "".join(ch * int(n) for ch, n in re.findall(r"(\D)(\d+)", s))

assert decompress(compress("aaabccdddd")) == "aaabccdddd"
print(decompress("a3b1c2"))
```

OUTPUT

```
aaabcc
```

P10 پرانتزهای متوازن Balanced brackets

O(n)

الگوی stack: پرانتز باز را `push` کن؛ با دیدن پرانتز بسته، آخرین باز باید جفتش باشد. در پایان، stack باید خالی باشد.

```
def is_balanced(s: str) -> bool:
    pairs = {")": "(", "]": "[", "}": "{"}
    stack = []
    for ch in s:
        if ch in "([{":
            stack.append(ch)
        elif ch in pairs:
            if not stack or stack.pop() != pairs[ch]:
                return False
    return not stack                          # leftovers -> not balanced

for t in ["(a[b]{c})", "(]", "((", ""]:
    print(repr(t), is_balanced(t))
```

OUTPUT

```
'(a[b]{c})' True
'(]' False
'((' False
'' True
```

PART 07

مسئله‌های عددی

P11 FizzBuzz — The classic warm-up

O(n)

از 1 تا n: مضرب 3 ← Fizz، مضرب 5 ← Buzz، مضرب هر دو ← FizzBuzz، بقیه ← خود عدد.

```python
def fizzbuzz(n: int) -> list[str]:
    result = []
    for i in range(1, n + 1):
        if i % 15 == 0:                    # check 15 FIRST
            result.append("FizzBuzz")
        elif i % 3 == 0:
            result.append("Fizz")
        elif i % 5 == 0:
            result.append("Buzz")
        else:
            result.append(str(i))
    return result

print(", ".join(fizzbuzz(15)))
```

```
OUTPUT
1, 2, Fizz, 4, Buzz, Fizz, 7, 8, Fizz, Buzz, 11, Fizz, 13, 14, FizzBuzz
```

اشتباه رایج: اگر شرط 15 را آخر بگذاری، هیچ‌وقت اجرا نمی‌شود، چون 15 قبلش در شرط «مضرب 3» گیر می‌کند. همیشه خاص‌ترین شرط را اول بنویس.

P12 اعداد اول — Prime check & sieve

O(√n)

```python
def is_prime(n: int) -> bool:
    if n < 2:
        return False
    i = 2
    while i * i <= n:                      # only up to sqrt(n)
        if n % i == 0:
            return False
        i += 1
    return True

def primes_up_to(n: int) -> list[int]:
    """Sieve of Eratosthenes."""
    if n < 2:
        return []
    is_p = [True] * (n + 1)
    is_p[0] = is_p[1] = False
    for i in range(2, int(n ** 0.5) + 1):
        if is_p[i]:
            for j in range(i * i, n + 1, i):
                is_p[j] = False
    return [i for i, p in enumerate(is_p) if p]

print([n for n in range(20) if is_prime(n)])
print(primes_up_to(30))
```

```
OUTPUT
[2, 3, 5, 7, 11, 13, 17, 19]
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
```

کافی است تا `√n` چک کنی: اگر `n = a × b` باشد، یکی از a یا b حتماً از `√n` کوچک‌تر یا مساوی آن است. غربال اراتستن (sieve) برای «همه‌ی اعداد اول تا n» است.

P13 فیبوناچی Fibonacci

`O(n)`

```python
from functools import lru_cache

def fib_list(n: int) -> list[int]:
    """First n Fibonacci numbers (iterative, O(n))."""
    seq = []
    a, b = 0, 1
    for _ in range(n):
        seq.append(a)
        a, b = b, a + b
    return seq

@lru_cache(maxsize=None)                    # memoization
def fib(n: int) -> int:
    """n-th Fibonacci number (recursive)."""
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)

print(fib_list(10))
print(fib(50))
```

```
OUTPUT
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
12586269025
```

سوال احتمالی: «پیچیدگی نسخه‌ی بازگشتیِ ساده (بدون cache) چقدر است؟» — `O(2ⁿ)` ، چون هر مقدار بارها از نو حساب می‌شود. `lru_cache` نتیجه‌ها را نگه می‌دارد و آن را `O(n)` می‌کند. محدودیت عمق بازگشت در پایتون حدود 1000 است، پس برای n بزرگ نسخه‌ی حلقه‌ای بهتر است.

P14 فاکتوریل Factorial (loop & recursion)

O(n)

```python
import math

def factorial(n: int) -> int:
    if n < 0:
        raise ValueError("n must be >= 0")
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

def factorial_rec(n: int) -> int:
    return 1 if n <= 1 else n * factorial_rec(n - 1)

assert factorial(5) == factorial_rec(5) == math.factorial(5) == 120
assert factorial(0) == 1
print(factorial(10))
```

```
OUTPUT
3628800
```

هر تابع بازگشتی دو جزء دارد: **حالت پایه** (base case) که بازگشت را تمام می‌کند (`n <= 1`) و **قدم بازگشتی** که مسئله را کوچک‌تر می‌کند. یادت نرود `0! = 1` .

P15 کار با رقم‌ها Sum of digits, reverse an integer

O(digits)

```python
def sum_digits(n: int) -> int:
    n = abs(n)
    total = 0
    while n > 0:
        total += n % 10                    # last digit
        n //= 10                           # drop last digit
    return total

def reverse_int(n: int) -> int:
    sign = -1 if n < 0 else 1
    return sign * int(str(abs(n))[::-1])

print(sum_digits(2026), sum(int(d) for d in str(2026)))
print(reverse_int(-1230))
print(math.gcd(12, 18))                    # greatest common divisor
```

```
OUTPUT
10 10
-321
6
```

دو ترفند اصلی: `n % 10` رقم آخر را می‌دهد و `n // 10` رقم آخر را حذف می‌کند. راه ساده‌تر هم تبدیل به رشته است: `str(n)` .

PART 08

مسئله‌های لیست

P16 بزرگ‌ترین و دومین بزرگ‌ترین Max & second max without max()

O(n)

مصاحبه‌گر می‌خواهد ببیند بدون توابع آماده می‌توانی حلقه بنویسی. حالت‌های مرزی: لیست خالی، همه‌ی اعضا برابر.

```python
def find_max(nums):
    if not nums:
        raise ValueError("empty list")
    best = nums[0]
    for n in nums[1:]:
        if n > best:
            best = n
    return best

def second_largest(nums):
    first = second = None
    for n in nums:
        if first is None or n > first:
            first, second = n, first
        elif n != first and (second is None or n > second):
            second = n
    return second

print(find_max([3, 7, 2]), second_largest([3, 7, 2, 7, 5]))
print(second_largest([4, 4]))                # no distinct second -> None
```

```
OUTPUT
7 5
None
```

P17 حذف تکراری‌ها با حفظ ترتیب Remove duplicates, keep order

O(n)

```python
def dedupe(items):
    seen, result = set(), []
    for x in items:
        if x not in seen:                    # set lookup is O(1)
            seen.add(x)
            result.append(x)
    return result

nums = [3, 1, 3, 2, 1]
dedupe(nums)               # → [3, 1, 2]
list(dict.fromkeys(nums))  # → [3, 1, 2]
sorted(set(nums))          # → [1, 2, 3]
```

`set` ترتیب را حفظ نمی‌کند. `dict.fromkeys` ترتیب را حفظ می‌کند (پایتون 3.7+). نسخه‌ی حلقه‌ای با `seen` را بلد باش؛ این الگو در خیلی از مسئله‌ها تکرار می‌شود.

P18 **Two Sum** Indices of two numbers that add up to target O(n)

معروف‌ترین سوال مصاحبه. اول راه `O(n²)` را بگو، بعد با دیکشنری `O(n)` کن: برای هر عدد، «عدد مکمل» را در دیکشنری عددهای دیده‌شده جست‌وجو کن.

```python
def two_sum_brute(nums, target):            # O(n^2)
    for i in range(len(nums)):
        for j in range(i + 1, len(nums)):
            if nums[i] + nums[j] == target:
                return i, j
    return None

def two_sum(nums, target):                  # O(n) with a dict
    seen = {}                               # value -> index
    for i, n in enumerate(nums):
        need = target - n
        if need in seen:
            return seen[need], i
        seen[n] = i
    return None

print(two_sum_brute([2, 7, 11, 15], 9), two_sum([2, 7, 11, 15], 9))
print(two_sum([3, 3], 6), two_sum([1, 2], 10))
```

```
OUTPUT
(0, 1) (0, 1)
(0, 1) None
```

"For each number I check whether its complement, target minus the number, is already in the dictionary. Dictionary lookups are O(1) on average, so the whole solution is O(n) time and O(n) space."

P19 **عدد گم‌شده** Missing number(s) O(n)

```python
def missing_number(nums):
    """nums holds 0..n with exactly one number missing."""
    n = len(nums)
    return n * (n + 1) // 2 - sum(nums)      # expected sum - real sum

def missing_numbers(nums, start, end):
    return sorted(set(range(start, end + 1)) - set(nums))

print(missing_number([3, 0, 1]))
print(missing_numbers([1, 2, 4, 7], 1, 7))
```

```
OUTPUT
2
[3, 5, 6]
```

فرمول گاوس: جمع 0 تا n برابر `n(n+1)/2` است؛ تفاوتش با جمع واقعی، همان عدد گم‌شده است. برای چند عدد گم‌شده، از تفاضل set استفاده کن.

P20 ادغام دو لیست مرتب Merge two sorted lists

O(n + m)

```python
def merge_sorted(a, b):
    i = j = 0
    result = []
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            result.append(a[i])
            i += 1
        else:
            result.append(b[j])
            j += 1
    result.extend(a[i:])                    # leftovers (one is empty)
    result.extend(b[j:])
    return result

print(merge_sorted([1, 4, 9], [2, 3, 10, 12]))
```

```
OUTPUT
[1, 2, 3, 4, 9, 10, 12]
```

الگوی دو اشاره‌گر. راه ساده‌تر `sorted(a + b)` است که `O((n+m) log(n+m))` است؛ اول همین را بگو، بعد نسخه‌ی بهتر را بنویس.

P21 صاف کردن لیست تودرتو Flatten a nested list

O(n)

```python
def flatten(items):
    result = []
    for x in items:
        if isinstance(x, list):
            result.extend(flatten(x))       # recursion
        else:
            result.append(x)
    return result

print(flatten([1, [2, [3, 4]], [], 5]))
print([x for row in [[1, 2], [3, 4]] for x in row])   # one level only
```

```
OUTPUT
[1, 2, 3, 4, 5]
[1, 2, 3, 4]
```

P22 چرخاندن و تکه‌تکه کردن Rotate a list / split into chunks

O(n)

```python
def rotate(nums, k):
    if not nums:
        return []
    k %= len(nums)                          # k may be bigger than len
    return nums[-k:] + nums[:-k]

def chunk(items, size):
    return [items[i:i + size] for i in range(0, len(items), size)]

print(rotate([1, 2, 3, 4, 5], 2), rotate([1, 2, 3], 3))
print(chunk([1, 2, 3, 4, 5, 6, 7], 3))
```

```
OUTPUT
[4, 5, 1, 2, 3] [1, 2, 3]
[[1, 2, 3], [4, 5, 6], [7]]
```

P23 تکراری‌ها و اشتراک Duplicates & common elements

O(n)

```python
from collections import Counter

def find_duplicates(items):
    return [x for x, c in Counter(items).items() if c > 1]

def common(a, b):
    return sorted(set(a) & set(b))

print(find_duplicates([1, 2, 3, 2, 4, 1]))
print(common([1, 2, 3, 4], [3, 4, 5]))
```

```
OUTPUT
[1, 2]
[3, 4]
```

P24 میانگین متحرک Moving average (sliding window)

O(n)

برای داده‌های سنسور و سرعت خودرو خیلی رایج است. راه ساده: برای هر پنجره `sum()` بگیر ← `O(n·k)` . راه بهتر: پنجره را «بلغزان»؛ یکی اضافه کن، یکی کم کن.

```python
def moving_average(values, window):
    if window <= 0 or window > len(values):
        raise ValueError("invalid window")
    window_sum = sum(values[:window])
    result = [window_sum / window]
    for i in range(window, len(values)):
        window_sum += values[i] - values[i - window]   # slide the window
        result.append(window_sum / window)
    return result

speeds = [50, 52, 55, 60, 58, 61]
print([round(x, 2) for x in moving_average(speeds, 3)])
```

```
OUTPUT
[52.33, 55.67, 57.67, 59.67]
```

P25 بیشترین سود خرید و فروش Best time to buy and sell (once)

O(n)

```python
def max_profit(prices):
    """Best profit from one buy followed by one sell."""
    lowest, best = float("inf"), 0
    for p in prices:
        lowest = min(lowest, p)              # cheapest buy so far
        best = max(best, p - lowest)         # sell today?
    return best

print(max_profit([7, 1, 5, 3, 6, 4]), max_profit([7, 6, 4, 3]))
```

```
OUTPUT
5 0
```

P26 جست‌وجوی دودویی Binary search

O(log n)

فقط روی لیست **مرتب**. هر بار نصف بازه را کنار می‌گذارد.

```python
import bisect

def binary_search(nums, target):
    """Index of target in the SORTED list nums, or -1."""
    lo, hi = 0, len(nums) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if nums[mid] == target:
            return mid
        if nums[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1

data = [2, 5, 8, 12, 16, 23, 38]
print(binary_search(data, 23), binary_search(data, 7))
print(bisect.bisect_left(data, 23))          # built-in helper
```

```
OUTPUT
5 -1
5
```

P27 پیاده‌سازی یک مرتب‌سازی Bubble sort

O(n²)

```python
def bubble_sort(nums):
    nums = nums[:]                           # don't modify the input
    n = len(nums)
    for i in range(n):
        swapped = False
        for j in range(n - 1 - i):
            if nums[j] > nums[j + 1]:
                nums[j], nums[j + 1] = nums[j + 1], nums[j]
                swapped = True
        if not swapped:                      # already sorted -> stop
            break
    return nums

print(bubble_sort([5, 1, 4, 2, 8]), bubble_sort([]))
```

```
OUTPUT
[1, 2, 4, 5, 8] []
```

سوال احتمالی: «Would you use this in production?» — نه؛ `sorted()` از الگوریتم Timsort استفاده می‌کند که `O(n log n)` و stable است و در C پیاده‌سازی شده. bubble sort فقط برای نشان دادن منطق است.

P28 **ماتریس** Transpose, row/column sums

O(rows·cols)

```python
m = [[1, 2, 3],
     [4, 5, 6]]
[list(row) for row in zip(*m)]  # → [[1, 4], [2, 5], [3, 6]]
[sum(row) for row in m]         # → [6, 15]
[sum(col) for col in zip(*m)]   # → [5, 7, 9]
max(max(row) for row in m)      # → 6
[[0] * 3 for _ in range(2)]     # → [[0, 0, 0], [0, 0, 0]]
```

`zip(*m)` سطرها را باز می‌کند و ستون‌ها را کنار هم می‌گذارد، یعنی ترانهاده. برای ساختن ماتریس خالی، حتماً از comprehension استفاده کن (بخش 11.4 را ببین).

PART 09

پردازش داده با پایتون خالص

برای نقش Projektmanagement & Entwicklung این محتمل‌ترین نوع سوال است: «یک لیست از دیکشنری‌ها (یا یک CSV کوچک) داریم؛ فیلتر کن، مرتب کن، گروه‌بندی کن، خلاصه کن.» همه‌ی مسئله‌های این بخش روی این دیتا کار می‌کنند:

```
cars = [
    {"model": "C200", "type": "petrol",   "price": 48_000,  "year": 2021},
    {"model": "EQE",  "type": "electric", "price": 79_000,  "year": 2023},
    {"model": "E300", "type": "hybrid",   "price": 65_000,  "year": 2022},
    {"model": "EQS",  "type": "electric", "price": 110_000, "year": 2022},
    {"model": "A180", "type": "petrol",   "price": 34_000,  "year": 2020},
]
```

P29 فیلتر، مرتب‌سازی، Top-N

Filter / sort / top-n on a list of dicts

O(n log n)

```
electric = [c["model"] for c in cars if c["type"] == "electric"]
print(electric)

by_price = sorted(cars, key=lambda c: c["price"], reverse=True)
print([c["model"] for c in by_price[:3]])        # top 3 most expensive

newest = max(cars, key=lambda c: c["year"])
print(newest["model"])

# sort by type (A-Z), then by price (high -> low)
ordered = sorted(cars, key=lambda c: (c["type"], -c["price"]))
print([c["model"] for c in ordered])
```

```
OUTPUT
['EQE', 'EQS']
['EQS', 'EQE', 'E300']
EQE
['EQS', 'EQE', 'E300', 'C200', 'A180']
```

برای مرتب‌سازی با چند معیار، `key` یک tuple برگرداند: اول بر اساس عضو اول، و در حالت مساوی بر اساس عضو دوم. منفی کردن یک عدد یعنی «این معیار نزولی باشد».

P30 گروه‌بندی و میانگین Group by + aggregate

O(n)

```python
from collections import defaultdict

prices_by_type = defaultdict(list)
for c in cars:
    prices_by_type[c["type"]].append(c["price"])

avg_by_type = {t: sum(p) / len(p) for t, p in prices_by_type.items()}
print(avg_by_type)

count_by_type = {}
for c in cars:
    count_by_type[c["type"]] = count_by_type.get(c["type"], 0) + 1
print(count_by_type)
```

```
OUTPUT
{'petrol': 41000.0, 'electric': 94500.0, 'hybrid': 65000.0}
{'petrol': 2, 'electric': 2, 'hybrid': 1}
```

نکته: این دقیقاً همان کاری است که `GROUP BY` در SQL یا `groupby` در pandas انجام می‌دهد (بخش 10.5). اگر یک الگو را از این جزوه حفظ کنی، این باشد.

P31 پردازش لاگ با خط‌های خراب Parse log lines robustly

O(n)

داده‌ی واقعی همیشه کثیف است. هدف: خط‌های خراب را رد کن، کرش نکن، و آخر بگو چند خط معتبر بود.

```python
log = """\
2026-10-01 08:00:01,V001,speed=52
2026-10-01 08:00:02,V002,speed=n/a
2026-10-01 08:00:03,V001,speed=61
bad line
2026-10-01 08:00:04,V002,speed=47
"""

def parse_line(line):
    parts = line.split(",")
    if len(parts) != 3:
        return None
    timestamp, vehicle, field = parts
    try:
        return timestamp, vehicle, float(field.split("=")[1])
    except (IndexError, ValueError):
        return None

records = [r for r in map(parse_line, log.splitlines()) if r]
max_speed = {}
for _, vehicle, speed in records:
    max_speed[vehicle] = max(speed, max_speed.get(vehicle, speed))
print(len(records), "valid records")
print(max_speed)
```

```
OUTPUT
3 valid records
{'V001': 61.0, 'V002': 47.0}
```

P32 تاریخ و زمان datetime basics

—

```python
from datetime import date, datetime, timedelta

start = datetime.strptime("2026-10-18 09:30", "%Y-%m-%d %H:%M")  # str->dt
end = start + timedelta(hours=2, minutes=15)
print(end.strftime("%d.%m.%Y %H:%M"))                             # dt->str
print((end - start).total_seconds() / 60, "minutes")

deadline = date(2026, 10, 18)
today = date(2026, 10, 17)
print((deadline - today).days, "day(s) left")
print(deadline.isoformat(), deadline.strftime("%A"), deadline.weekday())
```

```
OUTPUT
18.10.2026 11:45
135.0 minutes
1 day(s) left
2026-10-18 Sunday 6
```

Code	معنی	Code	معنی
`%Y`	سال 4 رقمی (2026)	`%H`	ساعت 00 تا 23
`%m`	ماه 01 تا 12	`%M`	دقیقه
`%d`	روز ماه	`%S`	ثانیه
`%A`	اسم روز هفته (Sunday)	`%b`	اسم کوتاه ماه (Oct)

برای یادآوری: `strptime` = **p**arse (رشته ← تاریخ) و `strftime` = **f**ormat (تاریخ ← رشته). تفاضل دو تاریخ یک `timedelta` است. `weekday()` برای دوشنبه 0 و برای یکشنبه 6 برمی‌گرداند.

P33 طولانی‌ترین رشته‌ی پشت‌سرهم Longest streak above a threshold

O(n)

```python
def longest_streak(values, threshold):
    """Longest run of consecutive values above threshold."""
    best = current = 0
    for v in values:
        current = current + 1 if v > threshold else 0
        best = max(best, current)
    return best

temps = [88, 92, 95, 91, 85, 97, 99, 101, 102, 80]
print(longest_streak(temps, 90))
```

```
OUTPUT
4
```

P34 تمیز کردن داده Clean & validate values

O(n)

مقدار خالی، `None`، غیرعددی و خارج از محدوده را حذف کن. اول از مصاحبه‌گر بپرس داده‌ی نامعتبر باید **حذف** شود، **جایگزین** شود یا **خطا** بدهد.

```python
raw = [" 12.5", "", None, "abc", "7", "-3", "1e2"]

def clean(values, low=0, high=100):
    cleaned = []
    for v in values:
        if v is None or not str(v).strip():
            continue                          # skip missing
        try:
            x = float(v)
        except ValueError:
            continue                          # skip not-a-number
        if low <= x <= high:                  # skip out-of-range
            cleaned.append(x)
    return cleaned

print(clean(raw))
```

```
OUTPUT
[12.5, 7.0, 100.0]
```

P35 دیکشنری تودرتو (شبیه JSON یک API) Nested data

O(n)

```python
order = {
    "id": 17,
    "customer": {"name": "Sara", "city": "Stuttgart"},
    "items": [{"part": "brake pad", "qty": 4, "price": 25.0},
              {"part": "oil filter", "qty": 1, "price": 12.5}],
}
total = sum(item["qty"] * item["price"] for item in order["items"])
print(order["customer"]["city"], total)
print(order.get("discount", {}).get("code", "none"))   # safe nested get
```

```
OUTPUT
Stuttgart 112.5
none
```

PART 10

pandas در 10 دقیقه

اگر تسک لایو «تحلیل داده» باشد، احتمالاً اجازه‌ی استفاده از pandas داری. اگر مطمئن نیستی، بپرس: "Can I use pandas, or should I solve it in plain Python?". دیتای این بخش یک لیست تسک‌های پروژه است.

> **نکته‌ی نسخه:** این خروجی‌ها از pandas 3.0 هستند. اگر روی سیستم مصاحبه pandas 2.x باشد، به جای نوع `str` نوع `object` می‌بینی؛ بقیه‌ی کدها یکسان کار می‌کنند.

10.1 ساختن و خواندن DataFrame

```python
import pandas as pd

df = pd.DataFrame({
    "task":    ["Spec", "Design", "Build", "Test", "Docs", "Review"],
    "owner":   ["Ali", "Sara", "Ali", "Mia", "Sara", "Mia"],
    "status":  ["done", "done", "open", "open", "done", None],
    "planned": [8, 16, 40, 24, 6, 4],
    "actual":  [10, 15, 52, None, 6, 5],
    "due":     ["2026-10-01", "2026-10-05", "2026-10-20",
                "2026-10-10", "2026-10-12", "2026-10-15"],
})
df.to_csv("tasks.csv", index=False)
df = pd.read_csv("tasks.csv")        # in the interview you start here
print(df.head(3))
print(df.shape)
```

```
OUTPUT
     task owner status  planned  actual         due
0    Spec   Ali   done        8    10.0  2026-10-01
1  Design  Sara   done       16    15.0  2026-10-05
2   Build   Ali   open       40    52.0  2026-10-20
(6, 6)
```

10.2 شناختن داده (همیشه اول این کار را بکن)

```
print(df.dtypes)
print(df.isna().sum()[lambda s: s > 0])      # only columns with gaps
print(df["owner"].value_counts())
print(df["planned"].describe()[["mean", "min", "max"]])
```

```
OUTPUT
task           str
owner          str
status         str
planned      int64
actual     float64
due            str
dtype: object
status    1
actual    1
dtype: int64
owner
Ali     2
Sara    2
Mia     2
Name: count, dtype: int64
mean    16.333333
min      4.000000
max     40.000000
Name: planned, dtype: float64
```

دستورهای دیگر: `df.info()` (نوع ستون‌ها و تعداد مقادیر غیرخالی)، `df.describe()` (آمار همه‌ی ستون‌های عددی)، `df.columns`، `df.tail()`.

10.3 انتخاب و فیلتر

```
print(df["task"].tolist())
print(df.loc[df["owner"] == "Ali", ["task", "planned"]])
print(df.iloc[0]["task"], df.loc[2, "owner"])
print(df[(df["status"] == "open") & (df["planned"] > 30)]["task"].tolist())
print(df[df["owner"].isin(["Mia", "Sara"])].shape)
```

```
OUTPUT
['Spec', 'Design', 'Build', 'Test', 'Docs', 'Review']
    task  planned
0   Spec        8
2  Build       40
Spec Ali
['Build']
(4, 6)
```

حواست باشد: برای فیلتر چندشرطی از `&`، `|` و `~` استفاده کن، نه `and/or/not`، و هر شرط را داخل پرانتز بگذار. `loc` با **اسم** سطر/ستون کار می‌کند و `iloc` با **شماره‌ی** موقعیت.

10.4 تمیزکاری و ستون جدید

```
df["status"] = df["status"].fillna("unknown")
df["actual"] = df["actual"].fillna(df["planned"])     # assume on plan
df["due"] = pd.to_datetime(df["due"])
df["overrun_pct"] = ((df["actual"] - df["planned"])
                     / df["planned"] * 100).round(1)
print(df[["task", "status", "actual", "overrun_pct"]])
```

```
OUTPUT
     task   status  actual  overrun_pct
0    Spec     done    10.0         25.0
1  Design     done    15.0         -6.2
2   Build     open    52.0         30.0
3    Test     open    24.0          0.0
4    Docs     done     6.0          0.0
5  Review  unknown     5.0         25.0
```

10.5 groupby و agg

```
summary = (df.groupby("owner")
             .agg(tasks=("task", "count"),
                  planned=("planned", "sum"),
                  actual=("actual", "sum"))
             .sort_values("actual", ascending=False))
print(summary)
print(df.groupby("status")["planned"].mean().round(1))
```

```
OUTPUT
       tasks  planned  actual
owner
Ali        2       48    62.0
Mia        2       28    29.0
Sara       2       22    21.0
status
done       10.0
open       32.0
unknown     4.0
Name: planned, dtype: float64
```

10.6 کار با تاریخ

```
today = pd.Timestamp("2026-10-17")
overdue = df[(df["status"] != "done") & (df["due"] < today)]
print(overdue[["task", "owner", "due"]])

df["days_left"] = (df["due"] - today).dt.days
df["weekday"] = df["due"].dt.day_name()
upcoming = df[(df["status"] != "done") & (df["days_left"] >= 0)]
print(upcoming[["task", "days_left", "weekday"]])
```

```
OUTPUT
     task owner        due
3    Test   Mia 2026-10-10
5  Review   Mia 2026-10-15
    task  days_left  weekday
2  Build          3  Tuesday
```

10.7 merge، apply و pivot_table

```
df["size"] = df["planned"].apply(lambda h: "large" if h >= 20 else "small")
teams = pd.DataFrame({"owner": ["Ali", "Sara", "Mia"],
                      "team": ["Dev", "UX", "QA"]})
merged = df.merge(teams, on="owner", how="left")       # like SQL JOIN
print(merged[["task", "owner", "team", "size"]].head(4))
print(pd.pivot_table(merged, index="team", columns="status",
                     values="planned", aggfunc="sum", fill_value=0))
```

OUTPUT

```
     task owner team   size
0    Spec   Ali  Dev  small
1  Design  Sara   UX  small
2   Build   Ali  Dev  large
3    Test   Mia   QA  large
status  done  open  unknown
team
Dev        8    40        0
QA         0    24        4
UX        22     0        0
```

می‌خواهم…	pandas	می‌خواهم…	pandas
CSV بخوانم/بنویسم	`pd.read_csv(p)` / `df.to_csv(p, index=False)`	تکراری‌ها را حذف کنم	`df.drop_duplicates()`
مرتب کنم	`df.sort_values("col", ascending=False)`	نوع ستون را عوض کنم	`df["c"].astype(int)`
شمارش هر مقدار	`df["c"].value_counts()`	اسم ستون عوض کنم	`df.rename(columns={"a": "b"})`
مقدار خالی	`isna()`، `fillna(v)`، `dropna()`	N سطر بزرگ‌تر	`df.nlargest(3, "col")`
ستون حذف کنم	`df.drop(columns=["c"])`	مقادیر یکتا	`df["c"].unique()`

PART 11

تست، Big-O و اشتباهات رایج

11.1 تست سریع با `assert`

```python
def normalize_plate(plate: str) -> str:
    """' s-mb 1234 ' -> 'S-MB 1234'"""
    return " ".join(plate.upper().split())

assert normalize_plate("s-mb 1234 ") == "S-MB 1234"
assert normalize_plate("  b  ab 12") == "B AB 12"     # extra spaces
assert normalize_plate("") == ""                      # edge case
print("all tests passed")
```

```
OUTPUT
all tests passed
```

چک‌لیست حالت‌های مرزی: ورودی خالی · فقط یک عضو · همه‌ی اعضا یکسان · مقادیر تکراری · صفر و عدد منفی · `None` یا نوع اشتباه · حروف بزرگ/کوچک · فاصله‌ی اضافه · ورودی خیلی بزرگ.

11.2 تست با pytest (اگر بپرسند «چطور تستش می‌کنی؟»)

```python
# file: test_stats.py      run with:  pytest -q
import pytest

def average(values):
    if not values:
        raise ValueError("empty")
    return sum(values) / len(values)

def test_average_normal():
    assert average([1, 2, 3]) == 2

def test_average_float():
    assert average([0.1, 0.2]) == pytest.approx(0.15)

def test_average_empty():
    with pytest.raises(ValueError):
        average([])
```

فایل باید با `test_` شروع شود و تابع‌ها هم همین‌طور؛ اجرا با `pytest -q` . برای مقایسه‌ی عدد اعشاری از `pytest.approx` و برای تست خطا از `pytest.raises` استفاده کن.

11.3 Big-O در یک نگاه

Time	Operation	Time	Operation
O(1) avg	`x in set`, `d[key]`, `s.add(x)`	O(1)	`lst[i]`, `lst.append(x)`, `lst.pop()`
O(1)	`deque.popleft()`, `appendleft`	O(n)	`x in lst`, `lst.remove(x)`, `lst.index(x)`
O(n log n)	`sorted(lst)`, `lst.sort()`	O(n)	`lst.insert(0, x)`, `lst.pop(0)`
O(n²)	two nested loops	O(n)	one loop over n items
O(n²) total	`s += ch` in a loop (strings)	O(log n)	binary search

> **"The time complexity is O(n) because I go through the list once, and the space complexity is O(n) because of the dictionary."**

11.4 اشتباهات رایج (و راه درستشان)

1) آرگومان پیش‌فرض mutable: مقدار پیش‌فرض فقط یک‌بار ساخته می‌شود و بین همه‌ی فراخوانی‌ها مشترک است.

```python
def add_item_bad(item, items=[]):            # BAD: one list for all calls
    items.append(item)
    return items

def add_item_good(item, items=None):         # GOOD
    if items is None:
        items = []
    items.append(item)
    return items

print(add_item_bad("a"), add_item_bad("b"))      # surprise!
print(add_item_good("a"), add_item_good("b"))
```

```
OUTPUT
['a', 'b'] ['a', 'b']
['a'] ['b']
```

2) کپی نکردن: `b = a` کپی نمی‌سازد؛ فقط یک اسم دیگر برای همان لیست است.

```
a = [1, 2, 3]
b = a                                   # same object, not a copy!
b.append(4)
print(a)
c = a.copy()                            # or a[:] or list(a)
c.append(5)
print(a, c)

grid = [[0] * 2] * 2                    # BAD: rows share one list
grid[0][0] = 9
print(grid)
grid = [[0] * 2 for _ in range(2)]      # GOOD
grid[0][0] = 9
print(grid)
```

```
OUTPUT
[1, 2, 3, 4]
[1, 2, 3, 4] [1, 2, 3, 4, 5]
[[9, 0], [9, 0]]
[[9, 0], [0, 0]]
```

3) تغییر دادن لیست حین پیمایش آن:

```
nums = [1, 2, 2, 3, 4]
for n in nums:                          # BAD: removing while looping
    if n % 2 == 0:
        nums.remove(n)
print(nums)                             # a 2 survived!

nums = [1, 2, 2, 3, 4]
nums = [n for n in nums if n % 2 != 0]  # GOOD: build a new list
print(nums)
```

```
OUTPUT
[1, 2, 3]
[1, 3]
```

4) چند غافلگیری کوچک: `==` مقدار را مقایسه می‌کند و `is` یکی بودن شیء را (فقط برای `None` از `is` استفاده کن). اعداد اعشاری دقیق نیستند. `round` در پایتون «به نزدیک‌ترین عدد زوج» گرد می‌کند.

```
a, b = [1, 2], [1, 2]
a == b, a is b                  # → (True, False)
x = None
x is None                       # → True
0.1 + 0.2 == 0.3                # → False
abs((0.1 + 0.2) - 0.3) < 1e-9   # → True
7 / 2, 7 // 2, -7 // 2          # → (3.5, 3, -4)
round(2.5), round(3.5)          # → (2, 4)
"5" + str(5), int("5") + 5      # → ('55', 10)
```

5) رشته تغییرناپذیر است:

```python
s = "car"
try:
    s[0] = "b"
except TypeError as e:
    print("TypeError:", e)
s = "b" + s[1:]                          # build a new string
print(s)
```

```
OUTPUT
TypeError: 'str' object does not support item assignment
bar
```

بقیه‌ی اشتباه‌های پرتکرار:

- `range(n)` از 0 تا `n-1` است، نه تا n (off-by-one).
- فراموش کردن `return` ← تابع `None` برمی‌گرداند.
- `input()` همیشه رشته برمی‌گرداند ← `int(input())` .
- اسم متغیر را `list` ، `dict` ، `sum` ، `max` ، `id` یا `str` نگذار؛ تابع built-in را از کار می‌اندازی.
- قاطی کردن tab و space در تورفتگی ← همیشه 4 فاصله.

PART 12

پنج تمرین شبیه‌سازی مصاحبه

برای هر تمرین تایمر 20 دقیقه‌ای بگذار. اول **بدون نگاه به جواب** و با روش 6 مرحله‌ای بخش 1، بلند و به انگلیسی حلش کن؛ بعد با جواب مقایسه کن. این تمرین‌ها عمداً به کارِ «مدیریت پروژه و توسعه» در یک شرکت خودروسازی نزدیک‌اند.

M1 ردیاب تسک‌های پروژه — Task tracker class

OOP + dict

Write a **TaskTracker** class with: **add(title, owner, hours, due)** — reject duplicate titles and non-positive hours; **complete(title)**; **overdue(today)** — open tasks past their due date, oldest first; **workload()** — open hours per owner, highest first.

```
from dataclasses import dataclass
from datetime import date

@dataclass
class Task:
    title: str
    owner: str
    hours: float
    due: date
    done: bool = False

class TaskTracker:
    def __init__(self):
        self._tasks = {}                    # title -> Task

    def add(self, title, owner, hours, due):
        if title in self._tasks:
            raise ValueError(f"task {title!r} already exists")
        if hours <= 0:
            raise ValueError("hours must be positive")
        self._tasks[title] = Task(title, owner, hours, due)

    def complete(self, title):
        if title not in self._tasks:
            raise KeyError(f"unknown task {title!r}")
        self._tasks[title].done = True

    def open_tasks(self):
        return [t for t in self._tasks.values() if not t.done]

    def overdue(self, today):
        late = [t for t in self.open_tasks() if t.due < today]
        return sorted(late, key=lambda t: t.due)

    def workload(self):
        load = {}
        for t in self.open_tasks():
            load[t.owner] = load.get(t.owner, 0) + t.hours
        return dict(sorted(load.items(), key=lambda kv: -kv[1]))

tracker = TaskTracker()
tracker.add("Spec", "Ali", 8, date(2026, 10, 10))
tracker.add("Build", "Ali", 40, date(2026, 10, 30))
tracker.add("Test", "Mia", 16, date(2026, 10, 12))
```

```
tracker.add("Docs", "Sara", 4, date(2026, 10, 20))
tracker.complete("Spec")

today = date(2026, 10, 17)
print([t.title for t in tracker.overdue(today)])
print(tracker.workload())
```

```
OUTPUT
['Test']
{'Ali': 40, 'Mia': 16, 'Sara': 4}
```

O(n)

M2 خلاصه‌ی داده‌ی سنسور Sensor summary with bad values

Battery temperature readings may contain **None** and impossible spikes (> 60 °C). Return count, how many were dropped, min, max, average (2 decimals) and how many valid readings are above 25 °C. Return an empty dict if nothing is valid.

```
readings = [21.5, 22.0, None, 23.1, 85.0, 24.2, None, 25.0, 26.4, 27.9]

def summarize(values, limit=25.0, max_valid=60.0):
    valid = [v for v in values if v is not None and v <= max_valid]
    if not valid:
        return {}
    return {
        "count": len(valid),
        "dropped": len(values) - len(valid),
        "min": min(valid),
        "max": max(valid),
        "avg": round(sum(valid) / len(valid), 2),
        "above_limit": sum(1 for v in valid if v > limit),
    }

print(summarize(readings))
print(summarize([None, None]))
```

```
OUTPUT
{'count': 7, 'dropped': 3, 'min': 21.5, 'max': 27.9, 'avg': 24.3, 'above_limit': 2}
{}
```

M3 ادغام جلسه‌های هم‌پوشان Merge overlapping meetings

O(n log n)

Given meetings as ("HH:MM", "HH:MM") pairs in any order, return the merged busy blocks of the day.

```python
def to_minutes(t):
    h, m = map(int, t.split(":"))
    return h * 60 + m

def to_hhmm(minutes):
    return f"{minutes // 60:02d}:{minutes % 60:02d}"

def merge_meetings(meetings):
    """[('09:00', '10:00'), ...] -> merged busy blocks."""
    intervals = sorted((to_minutes(s), to_minutes(e)) for s, e in meetings)
    merged = []
    for start, end in intervals:
        if merged and start <= merged[-1][1]:        # overlaps last block
            merged[-1][1] = max(merged[-1][1], end)
        else:
            merged.append([start, end])
    return [(to_hhmm(s), to_hhmm(e)) for s, e in merged]

meetings = [("09:00", "10:00"), ("09:30", "11:00"),
            ("13:00", "13:30"), ("11:00", "11:15")]
print(merge_meetings(meetings))
```

```
OUTPUT
[('09:00', '11:15'), ('13:00', '13:30')]
```

ایده‌ی کلیدی: **اول بر اساس زمان شروع مرتب کن**؛ بعد هر جلسه یا با آخرین بلوک ادغام می‌شود یا یک بلوک جدید شروع می‌کند.
تبدیل زمان به دقیقه، مقایسه را ساده می‌کند.

M4 درآمد قطعات از CSV — Revenue per part from CSV

O(n log n)

From the orders CSV, compute revenue (qty × unit_price) per part, return the top 2 parts, and count rows that could not be parsed.

```python
import csv
import io
from collections import defaultdict

data = """order_id,part,qty,unit_price
1,brake pad,4,25.00
2,oil filter,2,12.50
3,brake pad,2,25.00
4,wiper,x,9.99
5,spark plug,8,7.25
"""

revenue = defaultdict(float)
skipped = 0
for row in csv.DictReader(io.StringIO(data)):    # or open("orders.csv")
    try:
        revenue[row["part"]] += int(row["qty"]) * float(row["unit_price"])
    except ValueError:
        skipped += 1

top = sorted(revenue.items(), key=lambda kv: kv[1], reverse=True)[:2]
print(top, "skipped:", skipped)
```

OUTPUT

```
[('brake pad', 150.0), ('spark plug', 58.0)] skipped: 1
```

M5 هزینه‌ی شارژ خودروی برقی EV charging session cost

O(1)

Cost = kWh × 0.59 € + a blocking fee of 0.10 € per minute after the first 240 minutes. Inputs: start and end as "YYYY-MM-DD HH:MM", and kWh. Reject negative durations.

```python
from datetime import datetime

FMT = "%Y-%m-%d %H:%M"

def charging_cost(start, end, kwh, price_per_kwh=0.59,
                  fee_per_min=0.10, free_minutes=240):
    """Energy cost + blocking fee after free_minutes."""
    minutes = (datetime.strptime(end, FMT)
               - datetime.strptime(start, FMT)).total_seconds() / 60
    if minutes < 0 or kwh < 0:
        raise ValueError("invalid session")
    blocking = max(0, minutes - free_minutes) * fee_per_min
    return round(kwh * price_per_kwh + blocking, 2)

print(charging_cost("2026-10-17 08:00", "2026-10-17 11:00", 45))
print(charging_cost("2026-10-17 08:00", "2026-10-17 13:30", 60))
```

```
OUTPUT
26.55
44.4
```

سوال‌هایی که بعد از تمرین‌ها احتمالاً می‌پرسند:

- "What if the file has 10 million rows?" — خط به خط بخوان (حلقه روی فایل)، همه را در حافظه نریز؛ در pandas از `chunksize` استفاده کن.
- "How would you test this?" — چند `assert` یا تست pytest برای حالت عادی، ورودی خالی و ورودی خراب.
- "How would you extend it?" — مثلاً پارامتر کردن عددهای ثابت (قیمت، آستانه)، اضافه کردن logging، یا جدا کردن خواندن داده از منطق محاسبه.

12.1 چک‌لیست شب قبل

- [ ] پایتون 3، VS Code و Jupyter نصب و تست شده‌اند (`python --version`)، و pandas هم نصب است.
- [ ] اشتراک صفحه در Teams / Zoom / Webex تست شده؛ فونت ادیتور بزرگ است و نوتیفیکیشن‌ها خاموش‌اند.
- [ ] این 12 الگو را یک‌بار **از حفظ** تایپ و اجرا کرده‌ای: شمارش با `dict.get` · `Counter` · گروه‌بندی با `defaultdict(list)` · `sorted(key=lambda)` با شرط · Two Sum با دیکشنری · comprehension · `try/except ValueError` · کلاس با `__init__` و `__repr__` · `@dataclass` · `csv.DictReader` · `datetime.strptime` · `df.groupby().agg()` .
- [ ] جمله‌های انگلیسی بخش 1.3 را چند بار بلند گفته‌ای.

12.2 حین مصاحبه

1. **آرام شروع کن:** سوال را با کلمات خودت تکرار کن و 1 تا 2 سوال روشن‌کننده بپرس.
2. **ساده شروع کن:** کد ساده‌ای که کار می‌کند، از کد بهینه‌ی نیمه‌کاره بهتر است.

3 **ساکت نمان:** حتی وقتی فکر می‌کنی، بگو به چه فکر می‌کنی.

4 **اگر گیر کردی:** مسئله را کوچک‌تر کن، شبه‌کد را به شکل کامنت بنویس، یا مستقیم hint بخواه. گیر کردن طبیعی است؛ نحوه‌ی برخوردت با آن مهم است.

5 **آخر کار:** تست کن، پیچیدگی را بگو، و بگو با وقت بیشتر چه چیزی را بهتر می‌کردی.

موفق باشی — Viel Erfolg!